HTTP + TLS PRACTICAL

Від TCP-байтів до HTTPS-дебагу: net, http, curl, tls, openssl

Заняття №3

Node.js Pro

Лектор
Кирило Орлов

r_d

СТЕК НА СЬОГОДНІ: net · http · curl · tls · openssl

Перша половина — HTTP руками	Друга половина — TLS поверх сокета
net — сокет, сирі байти	tls / https — шифруємо той самий TCP
http — llhttp-парсер, keep-alive, межі	x509 · trust chain · self-signed
методи · статус-коди · кеш (ETag/304)	openssl s_client — дебаг broken cert
curl -v — читаємо трафік «по дроту»	HTTPS = HTTP по зашифрованому TCP

НАСКРІЗНА НИТКА
Рівні абстракції: TCP-байти → HTTP-семантика → TLS. П'ять технологій — одна ідея: усе на одному сокеті.

ПЛАН ЗАНЯТТЯ



- HTTP руками: net, сирі байти, request-line
- Методи, статус-коди, кеш (ETag / 304)
- keep-alive, chunked, таймаут-дефолти
- TLS 1.3: три гарантії та handshake
- x509, trust chain, self-signed для дев
- Дебаг broken cert: openssl s_client; HSTS / SNI

6 КРОКІВ ЗА fetch('https://api…') — НЕ МАГІЯ

01. DNS resolve

домен → IP (перший round-trip)

02. TCP handshake

SYN / SYN-ACK / ACK — ~1 RTT

03. TLS 1.3 handshake

ClientHello + key share — ~1 RTT

04. HTTP request

GET /users — просто текст по TCP

05. Обробка + відповідь

сервер → 200 + заголовки + тіло

06. TLS close / keep-alive

закриваємо або тримаємо сокет (keep-alive)

АНАТОМІЯ СИРОГО HTTP/1.1 ЗАПИТУ

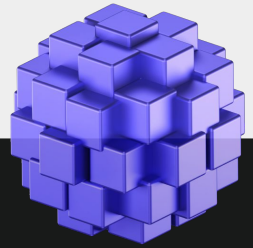
HTTP — формат листівки, не «технологія»

request-line: метод · шлях · версія (GET /users HTTP/1.1)

headers → службові позначки (Host, Accept, Authorization)

порожній рядок \r\n\r\n — кінець заголовків; далі body

Усе це — просто текст по TCP. Не JSON, не «магія»: метод, заголовки, порожній рядок, тіло.



1 REQUEST-LINE

```
GET /users HTTP/1.1
```

метод · шлях · версія

2 HEADERS

```
Host: api.example.com
Accept: application/json
Content-Length: 14
```

ключ: значення, по рядку на кожен

3 ПОРОЖНІЙ РЯДОК

```
\r\n\r\n
```

єдина межа — далі заголовків немає

4 BODY

```
{"name":"Ada"}
```

14 БАЙТІВ, не символів

СЕРВЕР НА net: 30 РЯДКІВ, ЩО БАЧАТЬ БАЙТИ

```
import net from 'node:net';
const server = net.createServer((socket) => {
  let buf = '';
  socket.on('data', (chunk) => {
    buf += chunk.toString('latin1');
    const end = buf.indexOf('\r\n\r\n');           // кінець
заголовків
    if (end === -1) return;                         // ще не всі
headers
    const [reqLine, ...headerLines] = buf.slice(0,
end).split('\r\n');
    const [method, path] = reqLine.split(' ');      // "GET /
HTTP/1.1"
    const headers = Object.fromEntries(headerLines.map((l) => {
      const i = l.indexOf(':');
      return [l.slice(0, i).toLowerCase(), l.slice(i + 1).trim()];
    }));
    const body = path === '/' ? `raw net, host=${headers.host}\n` :
'not found\n';
    const status = path === '/' ? '200 OK' : '404 Not Found';
    socket.write(
      `HTTP/1.1 ${status}\r\n` +
      `Content-Type: text/plain\r\n` +
      `Content-Length: ${Buffer.byteLength(body)}\r\n` +
      `Connection: close\r\n\r\n` + body);
    socket.end();
  });
});
server.listen(3000);
```

Три пастки рівня — код їх обходить, і ось як

1 · чанки ≠ запити → врятував indexOf('\r\n\r\n'); pipelining усе одно зламає

2 · заголовки — це байти, тому latin1, а НЕ utf8

3 · Content-Length у БАЙТАХ, тому Buffer.byteLength, а не .length

curl -v → 200 / Content-Length: 29

урок: розумій рівень нижче, користуйся http



«ПОШТОВА ЛИСТІВКА»: HTTP-ЗАПИТ ЯК КОНВЕРТ

ЧАСТИНА HTTP

- request-line: GET /path HTTP/1.1
- headers (Host, Accept, Authorization)
- порожній рядок \r\n\r\n
- body · TCP · Content-Length

АНАЛОГІЯ (ЛИСТІВКА)

- адреса + що зробити
- службові позначки (від кого, крихке)
- лінія між службовим і текстом
- сам текст · пошта везе байти · Content-Length = скільки байтів тіла

Edge-тест: листівка прийшла двома шматками → отримувач склеює за порожнім рядком + Content-Length (це вміє модуль http; сирий net — ні).



МЕТОДИ: SAFE / IDEMPOTENT / CACHEABLE

Категорія	Safe / Idempotent / Cacheable	Семантика
GET	✅ safe · ✅ idem · ✅ cache	прочитати ресурс
POST	✕ safe · ✕ idem · ⚠️ cache рідко	створити / дія (НЕ idempotent)
PUT	✕ safe · ✅ idem · ✕ cache	замінити ресурс цілком
DELETE	✕ safe · ✅ idem · ✕ cache	видалити ресурс
PATCH / HEAD	PATCH ✕ safe · ✕ idem · ✕ cache | HEAD ✅ · ✅ · ✅	PATCH — часткова зміна · HEAD — самі заголовки, без тіла

СТАТУС-КОДИ: 5 КЛАСІВ — КОНТРАКТ ДЛЯ CDN І БАЛАНСУВАЛЬНИКА

5 класів: 1xx info · 2xx success · 3xx redirect · 4xx client · 5xx server

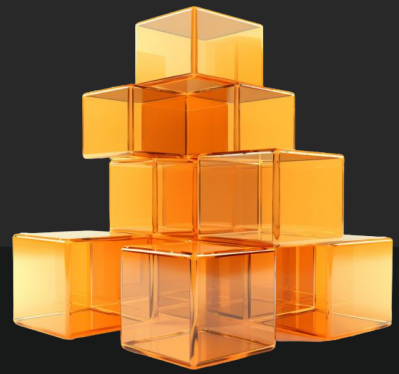
5xx = «зламався Я, не ти» — знімає провину з клієнта

401 = не знаю, хто ти · 403 = знаю, тобі не можна

400 = синтаксис зламаний · 422 = синтаксис ок, семантика ні

204 No Content краще за 200 з null-тілом

304 Not Modified — НЕ помилка; 429 + Retry-After → backoff



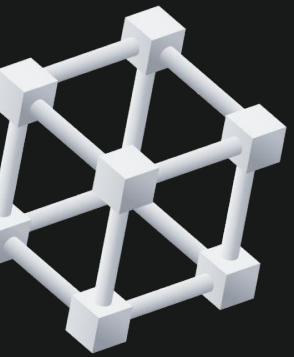


```
// ETag / 304 — умовний
 запит (демо b)

import http from 'node:http';
import { createHash } from 'node:crypto';
const body = JSON.stringify({ user: 'ada', role: 'admin' });
const etag = '"' + createHash('sha1').update(body).digest('base64').slice(0, 16) + '"';
http.createServer((req, res) => {
  if (req.headers['if-none-match'] === etag) {   // свіжа копія в клієнта
    res.writeHead(304, { ETag: etag });          // 304 — БЕЗ тіла
    return res.end();
  }
  res.writeHead(200, { 'Content-Type': 'application/json', ETag: etag, 'Cache-Control':
'no-cache' });
  res.end(body);
}).listen(3500);
```

CACHE-CONTROL НАСПРАВДІ + Vary (ТУТ УСІ ПОМИЛЯЮТЬСЯ)

- no-cache ≠ «не кешувати» → «кешуй, але перевалідуй»
- no-store = взагалі не зберігати (чутливі дані)
- max-age=N — свіжо без запиту; s-maxage — для CDN
- Vary = за якими заголовками(Accept-Language) різняться відповіді; забув → cache poisoning



ПЕРШИЙ ЗАПИТ

```
GET /users
```

```
200 OK
ETag: "a1b2c3"
+ повне тіло, 42 КБ
```

клієнт запам'ятав ETag

ДРУГИЙ ЗАПИТ

```
GET /users
If-None-Match: "a1b2c3"
```

не змінилось

304 Not Modified

0 байтів тіла

змінилось

200 OK

новий ETag + повне тіло

«ШТРИХ-КОД ВИДАННЯ»: ETag / КЕШУВАННЯ

ЧАСТИНА КОНЦЕПТУ

- ресурс (JSON / файл)
- ETag = штрих-код видання
- If-None-Match: <etag>
- збіг → 304 Not Modified (0 байт); зміна → 200 + новий ETag

АНАЛОГІЯ (БІБЛІОТЕКА)

- книга на полиці
- штрих-код конкретного видання
- «видання X ще чинне?»
- «так» — нову не несеш; нове видання — береш свіжу



```
// keep-alive + chunked: що http бере на себе

import http from 'node:http';
http.createServer((req, res) => {
  res.writeHead(200, { 'Content-Type': 'text/plain' }); // без Content-Length
  res.write('part 1\n');   // Node сам загорне у Transfer-Encoding: chunked
  res.write('part 2\n');
  res.end();               // фінальний нуль-chunk = «кінець»
}).listen(3000);
// keep-alive у HTTP/1.1 — дефолт; keepAliveTimeout=5000ms
```

ТАЙМАУТ-ДЕФОЛТИ + 502-ГОНКА З LB

requestTimeout · 300000 мс
увесь запит; захист від slowloris

headersTimeout · 60000 мс
час на прийом заголовків

keepAliveTimeout · 5000 мс
дефолт Node; idle keep-alive — ось де 502-гонка

502-гонка з LB — production-пастка
Балансувальник (LB) тримає сокет довше за keepAliveTimeout → Node закриває його під новий запит → випадкові 502, що не відтворюються локально. Фікс: keepAliveTimeout (60–75c) > idle апстріму; headersTimeout ще більший.

```
// TLS 1.3: 3 гарантії + handshake (1-RTT)

import tls from 'node:tls';
const socket = tls.connect(443, 'example.com', { servername: 'example.com' }, () => {
  console.log('authorized:', socket.authorized);        // пройшла перевірка chain?
  const cert = socket.getPeerCertificate();
  console.log(cert.subject.CN, cert.valid_to);          // кому видано, доки чинний
  if (!socket.authorized) console.error(socket.authorizationError); // CERT_HAS_EXPIRED
  socket.end();
});
```



TLS 1.3 HANDSHAKE: ОДИН ОБЕРТ, І ДАЛІ ВСЕ ЗАШИФРОВАНО



«РЕЖИМНИЙ ОФІС З ОХОРОНОЮ»: TLS ЯК АНАЛОГІЯ

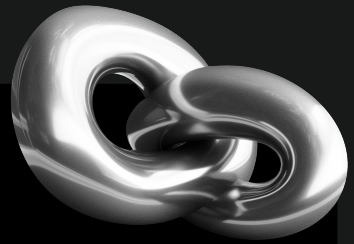
01. Cert = паспорт будівлі, виданий центром сертифікації (CA)

02. trust chain = МВС → облуправління → паспорт

03. session key (ECDHE) = перепуск лише на цей візит

04. forward secrecy = перепуск знищується на виході

05. authorized=false = не впізнали печатку / cert прострочено



```
// x509 + trust chain + self-signed для дев

openssl req -x509 -newkey rsa:2048 -nodes -keyout key.pem -out cert.pem -days 365 \
  -subj "/CN=localhost" -addext "subjectAltName=DNS:localhost"

import https from 'node:https';
import { readFileSync } from 'node:fs';

https.createServer(
  { cert: readFileSync('cert.pem'), key: readFileSync('key.pem') },
  (req, res) => res.end('tls ok\n')
).listen(3443);

// curl -k https://localhost:3443/ → 200. Без -k → SSL certificate problem.
```

SAN НЕ CN + REVOCATION ВМИРАЄ (CURRENT 2026)

SAN замінив CN у перевірці імені; без SAN → ERR_TLS_CERT_ALTNAME_INVALID

Проміжний cert забувають найчастіше → у клієнта verify-код 20

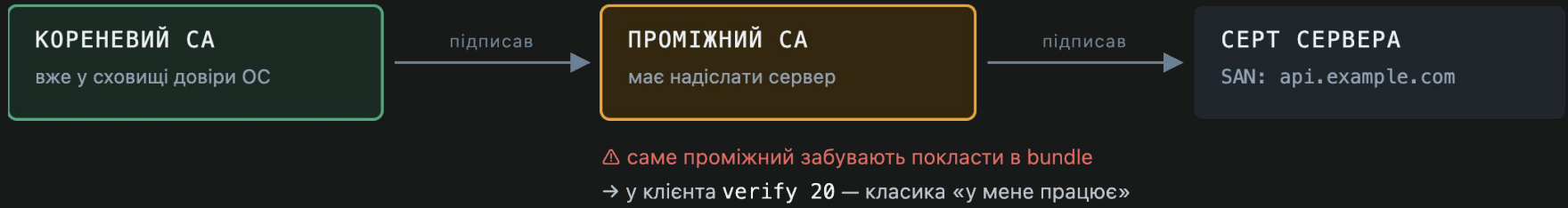
Let's Encrypt = ACME (RFC 8555): авто-видача, certbot / cert-manager

Let's Encrypt вимкнув OCSP (онлайн-перевірка відкликання, 06.08.2025) → CRL (список відкликаних)

З 15.01.2026 Let's Encrypt видає 6-денні серти без OCSP / CRL

Не покладайся на revocation — короткий lifetime + авто-поновлення

ЛАНЦЮГ ДОВІРИ — КОЖЕН ПІДПИСАВ НАСТУПНОГО

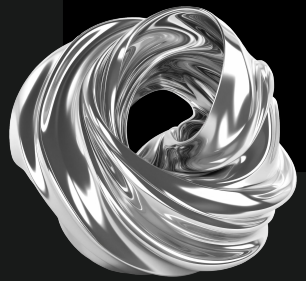


⚠ саме проміжний забувають покласти в bundle

→ у клієнта `verify 20` — класика «у мене працює»

```
// ⭐ ДЕМО: openssl s_client — читаємо broken cert номером

openssl s_client -connect localhost:3443 -servername localhost
#   Protocol           : TLSv1.3
#   Cipher is          : TLS_AES_256_GCM_SHA384
#   Verify return code : 18 (self-signed certificate)   ← діагноз
#   Negotiated TLS1.3 group: X25519MLKEM768             ← пост-квант, без коду
```



OPENSSL S_CLIENT: VERIFY-КОДИ І ЩО З НИМИ РОБИТИ

Код	Значення	Що робити
0	ok — chain валідний	у сховищі довіри
10	certificate has expired	поновити cert (перевір notAfter)
18	self-signed certificate	дев-cert; у проді — взяти від CA
20	unable to get local issuer	бракує intermediate у bundle (класика «у мене працює»)
21	unable to verify first cert	той самий intermediate-провал

HSTS + SNI: ЩО ВМИКАТИ І ЧОГО БОЯТИСЯ (2026)

HSTS (Strict-Transport-Security) = «рік ходи лише по HTTPS» → захист від SSL-stripping, підміни https на http

⚠ preload — це вшитий у браузери список; вийти з нього — місяці. Лише коли 100% інфри на HTTPS

SNI = багато сертів на один IP; ім ʼя домену в ClientHello летить відкрито, бо ключа ще нема — його ховає ECH (шифрований ClientHello)



ПІДСУМКИ + МІСТ ДО L#4

РІВНІ

три рівні на одному сокеті: TCP-байти → HTTP-семантика → TLS

HTTPS

HTTPS = HTTP по зашифрованому TCP, не окрема технологія

debug

broken cert = номер verify (18 / 10 / 20); s_client + curl / Node

ДЗ #3

ДЗ #3 (5 балів): raw net-сервер + HTTPS self-signed + s_client debug · ДАЛІ L#4 — Express і Fastify поверх того самого http

304 / http

304 — не помилка; http рятує від багів твого net-коду

Q&A

???



YOUR PLACE
TO GROW